

Пермский институт (филиал) федерального государственного
бюджетного образовательного учреждения высшего образования
«Российский экономический университет имени Г. В. Плеханова»

Кафедра информационных технологий и программирования

Практическая работа

Тема работы: Создание системы управления складом.

Выполнил: Коробейников Никита Дмитриевич

Группа: ИПс-31

Пермь 2026

Оглавление

Создание приложения и его настройка	3
Структура библиотеки Sklad.Core.....	6
Создание моделей данных	7
Создание сервисов для работы с данными.....	9
Создание базовых классов MVVM в папке ViewModels	10
Создание ViewModels	11
Структура основного WPF-проекта Sklad	13
Создание окон и представлений.....	14
Управление приложением с помощью мыши	16
Работа с накладными	17
Написание Unit-тестов	20
Запуск приложения и его тестирование	24
Заключение.....	31
Список использованных источников	33

Создание приложения и его настройка

Необходимо создать полноценное WPF-приложение в Visual Studio на языке программирования C# с функциональным и интуитивно понятным пользовательским интерфейсом для ведения учёта организаций, складов и номенклатуры товаров. Приложение строится на архитектурном паттерне MVVM (Model-View-ViewModel). Вместо базы данных используются коллекции — это обеспечивает чистое разделение ответственности.

Открываем Visual Studio. Выбираем шаблон — Приложение WPF (Рисунок 1).

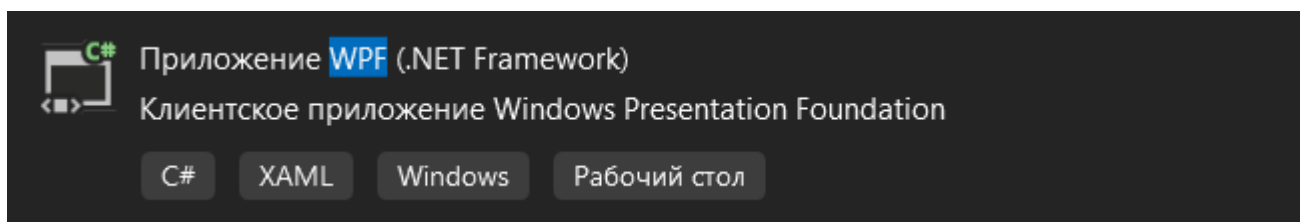


Рисунок-1

Задаём название проекта — Sklad, выбираем расположение и платформу .NET Framework 4.8 (Рисунок 2). Нажимаем «Создать».

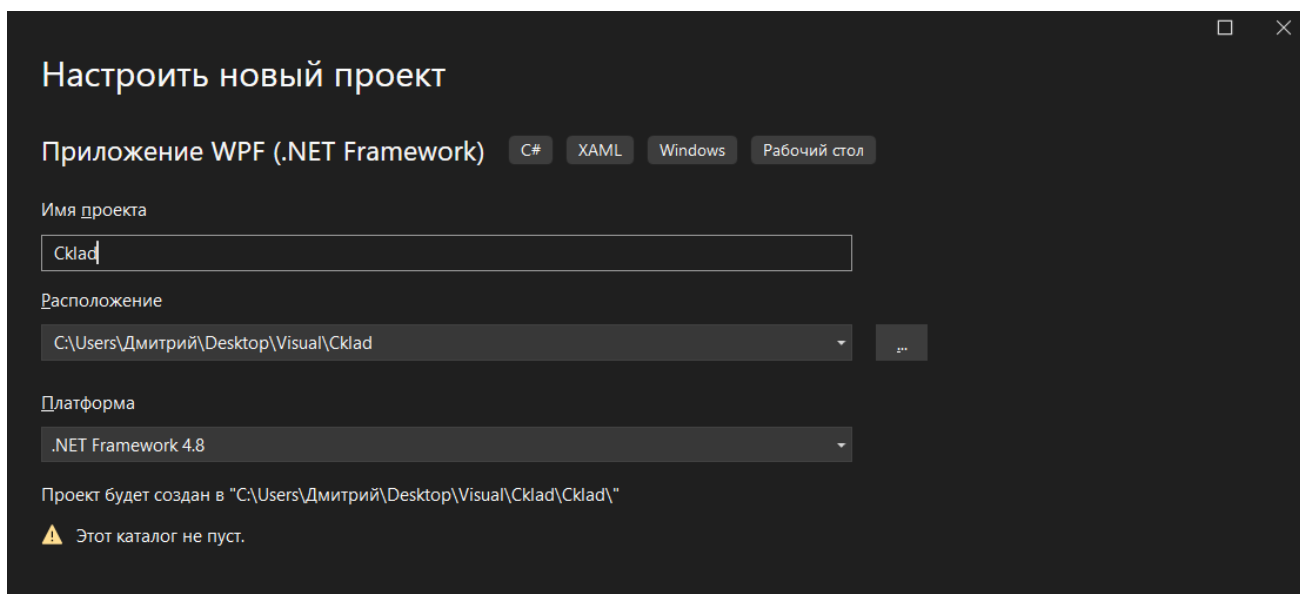


Рисунок-2

У нас создалось главное окно приложения. Теперь нужно создать библиотеку классов и проект для тестов.

Нажимаем ПКМ по надписи Решение «Склад» в обозревателе решений → «Добавить» → «Создать проект...» (Рисунок 3).

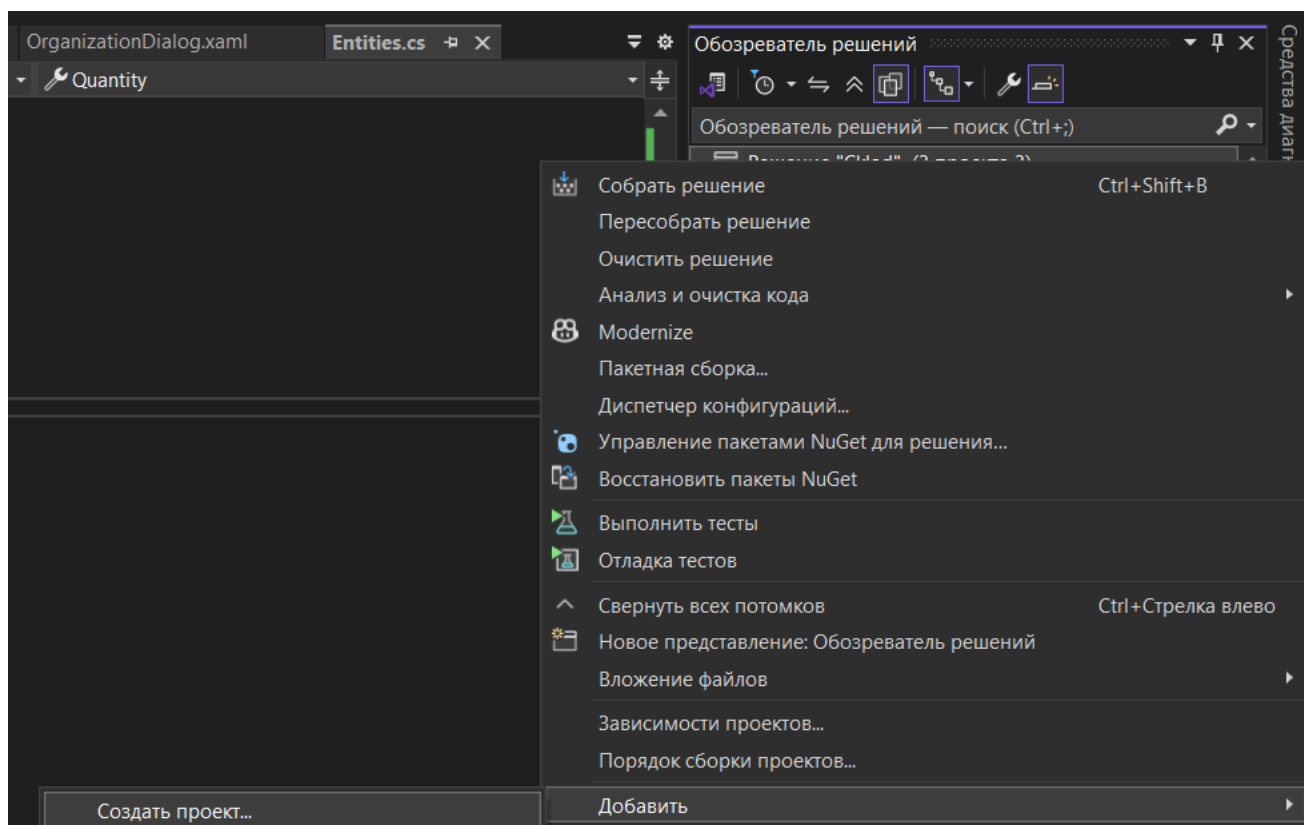


Рисунок-3

Выбираем шаблон «Библиотека классов (.NET Framework)» (Рисунок 4), называем проект Cklad.Core, платформа — .NET Framework 4.8.

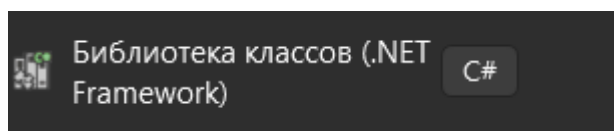


Рисунок-4

Далее создаём проект для тестов. По аналогии: Решение «Cklad» → «Добавить» → «Создать проект...» → шаблон «Проект модульного теста MSTest» (Рисунок 5). Называем его Cklad.Tests, платформа — та же .NET Framework 4.8.

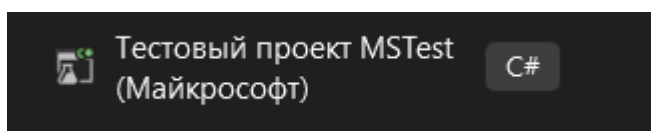


Рисунок-5

Теперь подключим ссылки между проектами. Нажимаем ПКМ по папке «Ссылки» в проекте Cklad → «Добавить ссылку» → «Проекты → Решение» → ставим галочку напротив Cklad.Core (Рисунок 6) → «ОК».

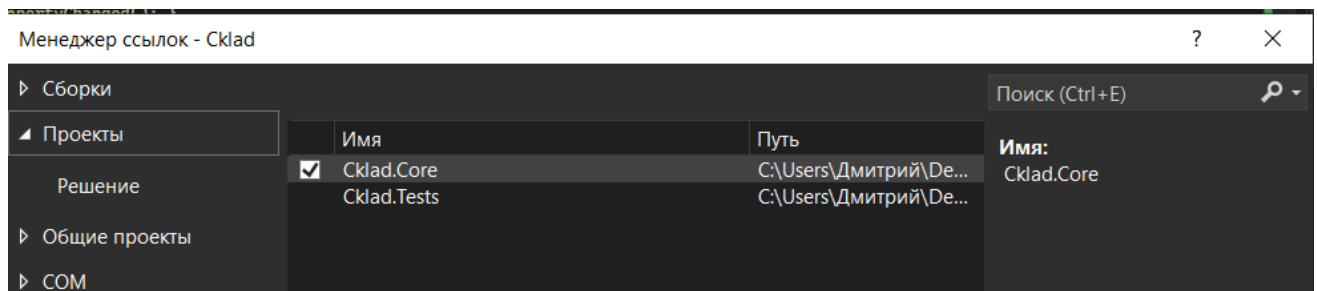


Рисунок-6

Повторяем то же самое для проекта Cklad.Tests — устанавливаем галочку напротив Cklad.Core. Ссылку на сам проект Cklad добавлять не нужно: тесты должны зависеть только от библиотеки, а не от WPF-проекта.

Готово. Наш проект теперь выглядит так: три проекта — главный (WPF), библиотека классов и тесты (Рисунок 7).

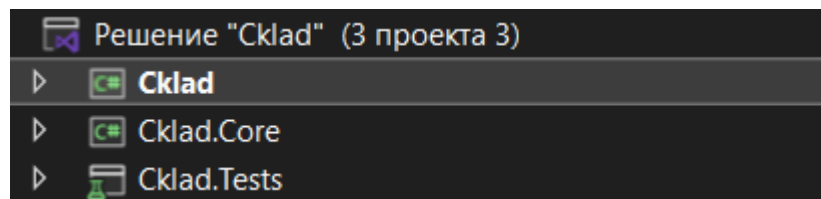


Рисунок-7

Структура библиотеки Sklad.Core

Вся бизнес-логика приложения вынесена в отдельный проект Sklad.Core. Это обеспечивает чистоту архитектуры и возможность тестирования логики независимо от графического интерфейса.

В проекте созданы три папки:

- Models — модели данных;
- Services — сервисы для работы с данными;
- ViewModels — ViewModels всех модулей.

В корне проекта автоматически находятся служебные файлы app.config и packages.config, созданные Visual Studio.

Общая структура библиотеки Sklad.Core показана на рисунке 8.

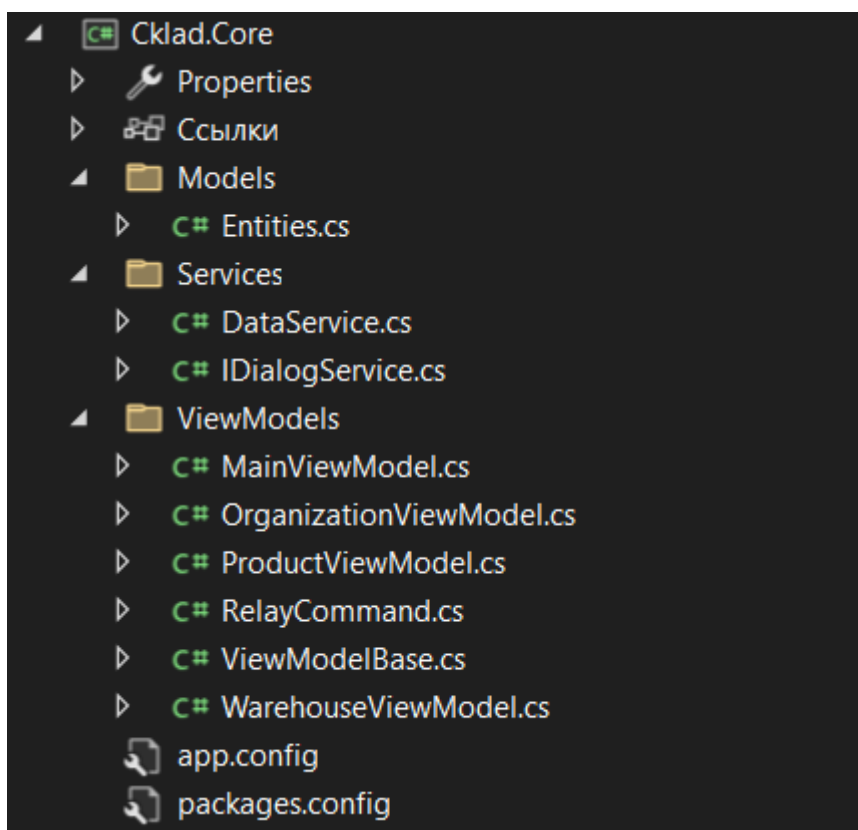


Рисунок-8

Создание моделей данных

В проекте Sklad.Core создаём папку Models: ПКМ по проекту Sklad.Core → «Добавить» → «Создать папку».

Внутри папки создаём файл Entities.cs: ПКМ по папке Models → «Добавить» → «Создать элемент...» → «Класс» → называем Entities.cs (Рисунок 9).

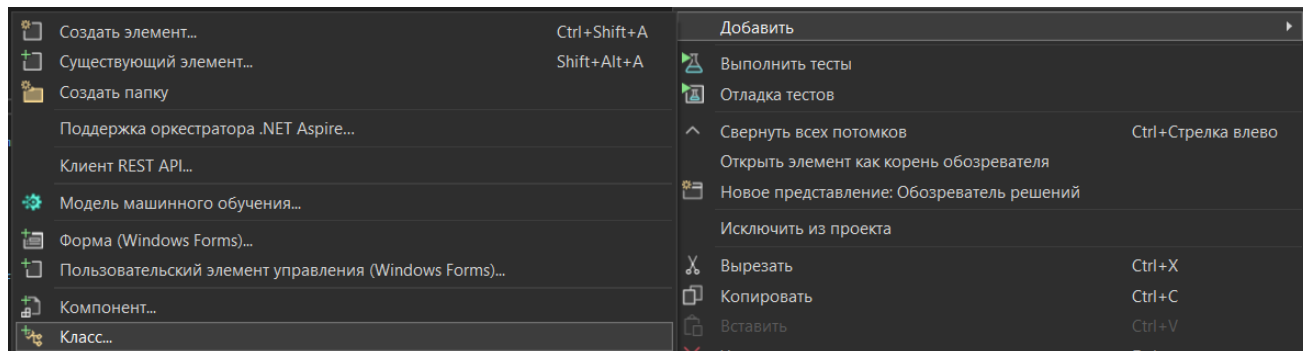


Рисунок-9

В этом единственном файле описываем все модели данных нашего приложения.

Organization

Модель организации (юридического лица). Содержит уникальный идентификатор Id (тип Guid) и название Name. Используется для хранения информации о компаниях-владельцах складов.

Warehouse

Модель склада. Включает идентификатор Id, ссылку на организацию OrganizationId, название Name и адрес Address. Каждый склад привязан к конкретной организации.

Product

Модель товара. Содержит идентификатор Id, ссылку на склад WarehouseId, наименование Name, категорию Category, производителя Manufacturer, поставщика Supplier, цену Price, остаток на складе Quantity, скидку в процентах DiscountPercent и путь к фотографии PhotoPath. Это основная единица складского учёта.

Invoice

Модель накладной. Содержит идентификатор Id, номер документа Number, ссылку на склад WarehouseId, дату создания Date, тип IsIncoming (приход или расход), статус Status (Проведена / Отменена) и коллекцию позиций Items. Также содержит вычисляемые свойства TypeDisplay (текстовое представление типа) и TotalSum (общая сумма накладной). Обеспечивает документооборот движения товаров партиями.

InvoiceItem

Модель позиции товара в накладной.

Содержит:

- ProductId (ссылка на товар),

- `ProductName` (название товара на момент проведения),
- `Quantity` (количество),
- `Price` (цена) и вычисляемое свойство `Total` (произведение количества на цену).

Реализует интерфейс `INotifyPropertyChanged` — это позволяет автоматически пересчитывать сумму при изменении количества прямо в диалоге накладной.

После создания папка `Models` содержит единственный файл `Entities.cs` (Рисунок 10).

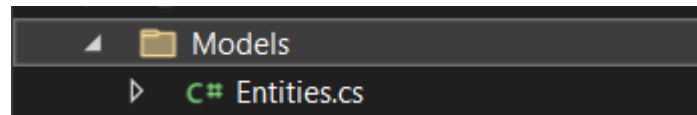


Рисунок-10

Создание сервисов для работы с данными

В том же проекте Sklad.Core создаём папку Services: ПКМ по проекту → «Добавить» → «Создать папку».

Создаём два файла.

4.1. IDialogService.cs

Интерфейс для всех диалоговых операций. Определяет методы:

- ShowInput() — показать поле ввода;
- Confirm() — запрос подтверждения;
- ShowMessage() — вывод сообщения;
- OpenFile() — выбор файла;
- ShowOrganizationDialog() — показ диалога организации;
- ShowWarehouseDialog() — показ диалога склада;
- ShowProductDialog() — показ диалога товара;
- ShowInvoiceDialog() — показ диалога создания накладной;
- ShowInvoicesListWindow() — показ окна со списком накладных.

Благодаря этому интерфейсу логика Sklad.Core не зависит от UI и может быть протестирована без участия пользователя.

DataService.cs

Хранилище всех данных приложения. Содержит ObservableCollection для каждой модели: Organizations, Warehouses, Products, Invoices. Реализован как синглтон — один экземпляр на всё приложение. В конструкторе заполняет коллекции тестовыми данными. Также содержит метод ResetInstance() — он используется в юнит-тестах для сброса состояния между тестами.

Папка Services после создания содержит два файла (Рисунок 11).

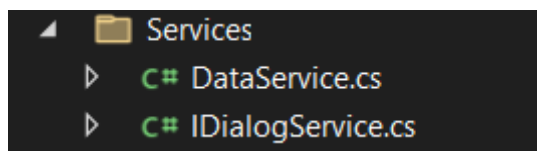


Рисунок-11

Создание базовых классов MVVM в папке ViewModels

Переходим к папке ViewModels в проекте Sklad.Core. Создаём её тем же способом — через «Добавить → Создать папку».

ViewModelBase.cs

Базовый класс для всех ViewModel. Реализует интерфейс INotifyPropertyChanged, что позволяет уведомлять интерфейс об изменении свойств. Содержит защищённый метод OnPropertyChanged() — он вызывается в сеттерах свойств, чтобы WPF перерисовал привязанные элементы.

RelayCommand.cs

Класс, реализующий интерфейс ICommand. Позволяет привязывать методы ViewModel к действиям пользователя — нажатие кнопки, выбор пункта меню. Содержит два делегата: _execute — метод, который выполняется при активации команды, и _canExecute — метод, проверяющий доступность команды. Дополнительно реализован метод RaiseCanExecuteChanged() для ручного пересчёта состояния кнопок и пунктов меню.

Создание ViewModels

В той же папке ViewModels создаём четыре ViewModel модулей:

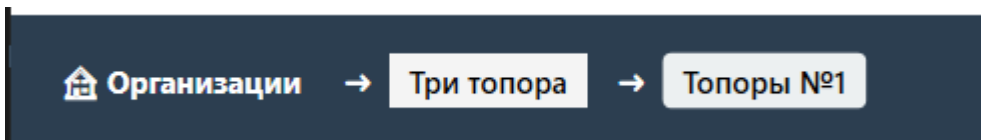
- MainViewModel.cs — главная ViewModel;
- OrganizationViewModel.cs — ViewModel организаций;
- WarehouseViewModel.cs — ViewModel складов;
- ProductViewModel.cs — ViewModel товаров.

MainViewModel.cs

Управляет текущим отображаемым модулем (CurrentViewModel), выбранной организацией (SelectedOrganization) и выбранным складом (SelectedWarehouse). Содержит три команды навигации:

- NavigateToOrganizationsCommand — возвращает к списку организаций;
- NavigateToWarehousesCommand — открывает список складов выбранной организации;
- NavigateToProductsCommand — открывает таблицу товаров выбранного склада.

Эти команды привязаны к панели навигации — она расположена в верхней части главного окна и отображает путь пользователя по иерархии данных:



OrganizationViewModel.cs

Содержит коллекцию организаций, выбранную организацию и команды: AddCommand, EditCommand, DeleteCommand, SelectCommand. Все команды, кроме Add, доступны только при выбранной строке — это реализовано через `_canExecute`. При выборе организации происходит переход к её складам.

WarehouseViewModel.cs

Фильтрует список складов по выбранной организации через LINQ:

```
var allWh = DataService.Instance.Warehouses;
Warehouses = new ObservableCollection<Warehouse>(
    allWh.Where(w => w.OrganizationId == _mainViewModel.SelectedOrganization.Id));
```

Содержит команды добавления, редактирования, удаления и выбора склада. Добавление и редактирование склада работают через отдельное диалоговое окно WarehouseDialog, которое позволяет ввести не только название, но и адрес склада. При выборе склада происходит переход к его товарам.

ProductViewModel.cs

Содержит коллекцию товаров выбранного склада, выбранный товар и команды:

- AddCommand, EditCommand, DeleteCommand — CRUD-операции над товарами;
- ImportCommand — импорт товаров из CSV и Excel;
- InvoiceInCommand, InvoiceOutCommand — оформление прихода и расхода;
- ShowInvoicesCommand — открытие списка накладных;
- UnpostInvoice() — отмена проведения накладной с восстановлением остатков.

В итоге папка ViewModels содержит шесть файлов (Рисунок 12).

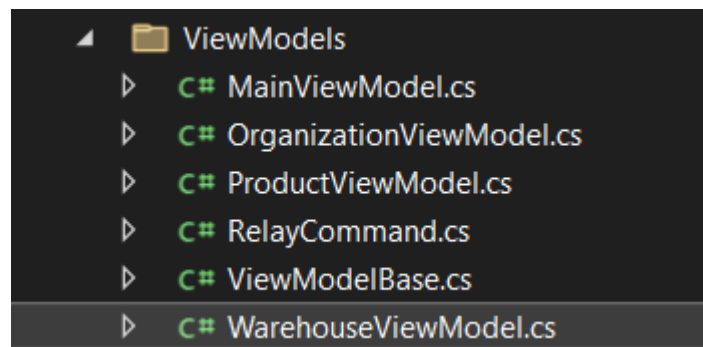


Рисунок-12

Структура основного WPF-проекта Cklad

В главном проекте Cklad созданы папки Views, Dialogs, Converters и Services. В корне проекта автоматически находятся App.xaml, MainWindow.xaml, файлы code-behind к ним, а также служебные app.config и packages.config.

Структура проекта Cklad показана на рисунке 13.

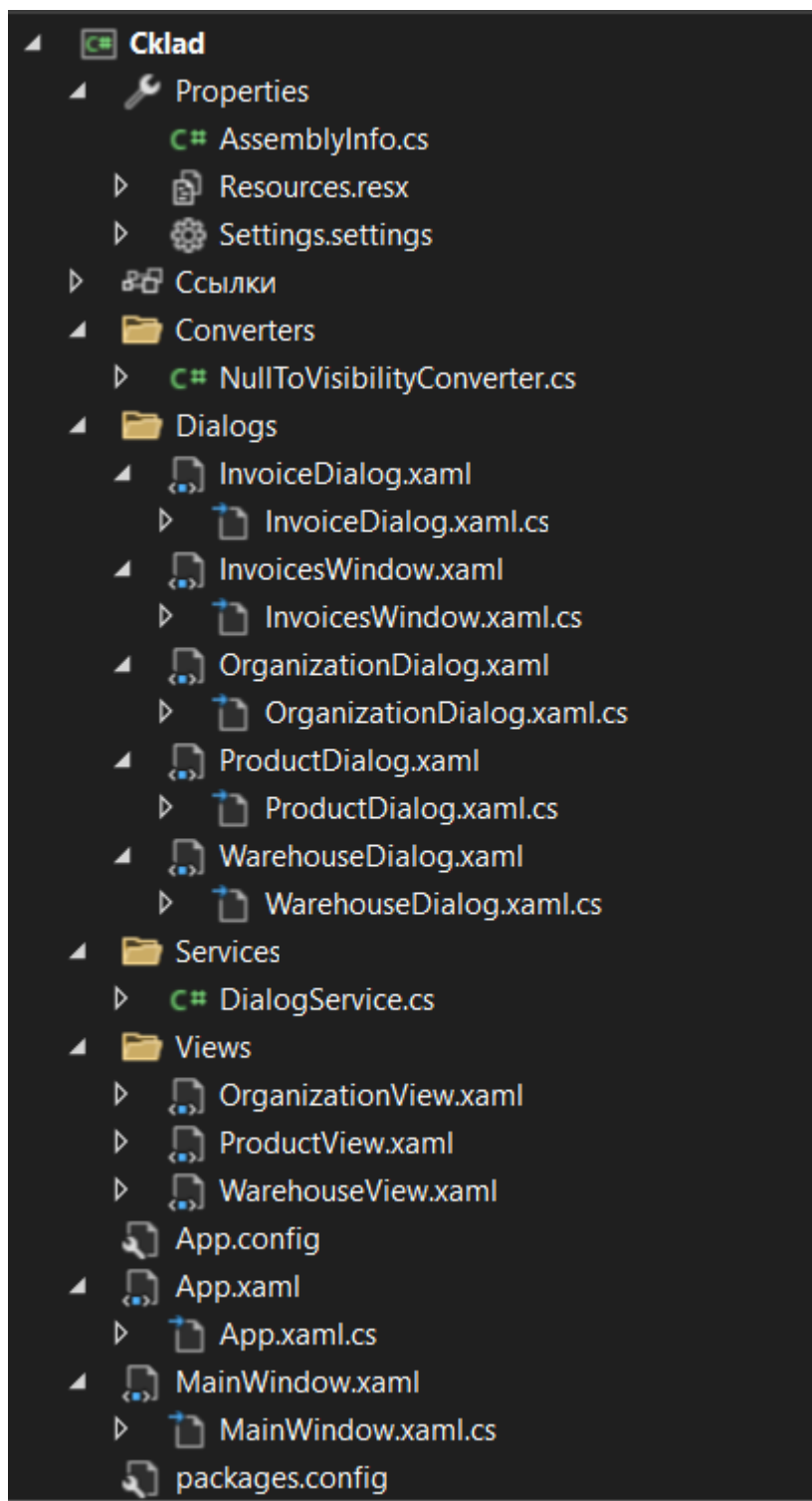


Рисунок-13

Создание окон и представлений

Папка Services (в проекте Cklad)

Здесь создаём DialogService.cs — реализацию интерфейса IDialogService. Именно он использует WPF-диалоги (MessageBox, OpenFileDialog) и вызывает окна из папки Dialogs. Метод ShowWarehouseDialog() открывает окно WarehouseDialog для ввода названия и адреса склада. Метод ShowInvoicesListWindow() открывает окно InvoicesWindow со списком накладных и подписывается на событие отмены проведения.

Папка Converters

Создаём NullToVisibilityConverter.cs — конвертер, который превращает null в Visibility.Collapsed, а не-null — в Visibility.Visible. Используется в панели навигации главного окна для показа и скрытия названий организации и склада.

Папка Dialogs

Создаём пять окон WPF («Добавить» → «Окно (WPF)»):

- OrganizationDialog.xaml — диалог создания и редактирования организации. Содержит поле ввода названия и кнопки «ОК» / «Отмена».
- WarehouseDialog.xaml — диалог создания и редактирования склада. Содержит два поля ввода: «Название склада» и «Адрес». Валидация не позволяет сохранить склад с пустым названием или адресом.
- ProductDialog.xaml — диалог создания и редактирования товара. Содержит поля всех характеристик товара и кнопку выбора фотографии.
- InvoiceDialog.xaml — диалог создания накладной прихода или расхода. Позволяет добавлять позиции с указанием товара и количества. Цена и сумма пересчитываются автоматически.
- InvoicesWindow.xaml — окно со списком всех накладных выбранного склада. Содержит кнопки «Детали», «Отменить проведение» и «Закрыть».

Папка Views

Создаём три пользовательских элемента управления («Добавить» → «Пользовательский элемент управления (WPF)»):

- OrganizationView.xaml — список организаций (ListView);
- WarehouseView.xaml — список складов (ListView);
- ProductView.xaml — таблица товаров (DataGrid).

App.xaml и MainWindow.xaml

- App.xaml содержит единую тему оформления (цвета, стили кнопок), шаблоны DataTemplate, связывающие ViewModels с Views, и регистрацию конвертера

NullToVisibilityConverter.

- MainWindow.xaml содержит панель навигации и ContentControl для отображения текущего модуля.

Папка Views после создания содержит три файла .xaml и три файла .xaml.cs (Рисунок 14).

Папка Dialogs содержит пять окон (Рисунок 15).

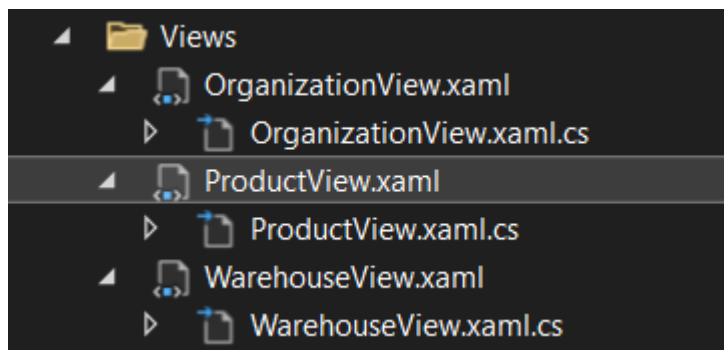


Рисунок-14

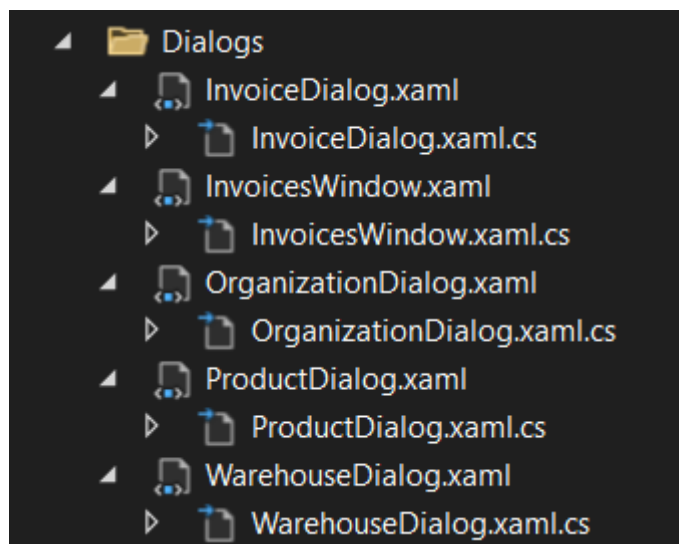


Рисунок-15

Управление приложением с помощью мыши

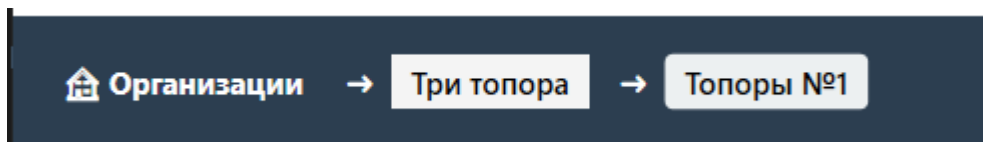
Особенностью приложения является управление преимущественно мышью — это делает интерфейс интуитивно понятным.

Клик правой кнопкой мыши (контекстное меню)

- По организациям — «Открыть склады», «Новая организация», «Изменить», «Удалить».
- По складам — «Открыть товары», «Новый склад», «Изменить», «Удалить».
- По товарам — «Редактировать», «Удалить», «Добавить товар», «Оформить приход», «Оформить расход», «Список накладных».

Панель навигации

В верхней части главного окна расположена панель навигации:



«Организации» — кнопка, возвращает к начальному списку организаций.

Кнопки на панели товаров

В модуле управления товарами оставлены четыре функциональные кнопки:

- Приход — открыть диалог создания накладной прихода (зелёного цвета).
- Расход — открыть диалог создания накладной расхода (оранжевого цвета).
- Импорт (CSV/Excel) — открыть диалог выбора файла для импорта товаров (жёлтого цвета).
- Накладные — открыть окно со списком всех накладных склада (синего цвета).

Остальные действия (добавление, редактирование, удаление товара) вынесены в контекстное меню ПКМ.

Работа с накладными

Реализован полный документооборот накладных.

Создание накладной

При нажатии кнопки Приход или Расход открывается диалоговое окно InvoiceDialog (Рисунок 16).

The screenshot shows a window titled "Накладная" (Invoice). It contains the following elements:

- A dropdown menu for "Тип накладной:" (Invoice type) with "Приход" (Receipt) selected.
- A date picker for "Дата:" (Date) showing "Выбор даты" (Choose date) and the date "15".
- Buttons for "Товары в накладной:" (Items in invoice): "+ Добавить строку" (Add line) and "X Удалить строку" (Delete line).
- A table with the following columns: "Товар" (Goods), "Количество" (Quantity), "Цена" (Price), and "Сумма" (Sum). The first row shows a quantity of "1", a price of "\$0.00", and a sum of "\$0.00".
- Buttons for "OK" and "Отмена" (Cancel) at the bottom right.

Рисунок-16

В окне:

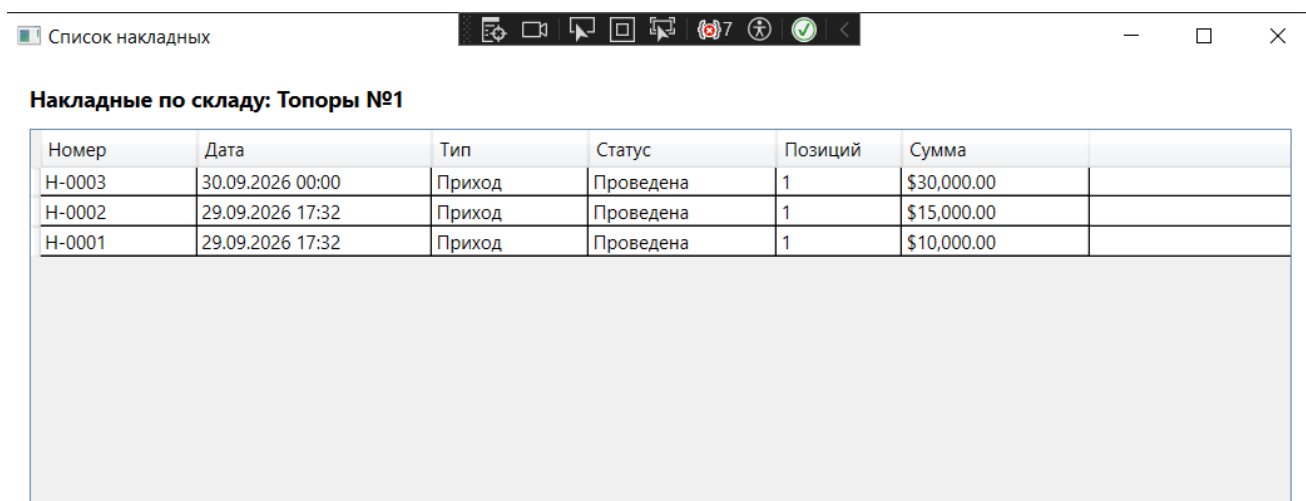
- выбирается тип накладной (Приход / Расход);
- указывается дата;
- добавляются позиции через кнопку «+Добавить строку»;
- для каждой позиции выбирается товар из выпадающего списка и вводится количество;
- цена и сумма подставляются автоматически.

После нажатия ОК:

- Проверяется достаточность товара при расходе.
- Остатки на складе изменяются (увеличиваются при приходе, уменьшаются при расходе).
- Накладная сохраняется в DataService.Invoices с автоматически сгенерированным номером (Н-0001, Н-0002...).
- Пользователю выводится сообщение об успешном проведении.

Просмотр списка накладных

При нажатии кнопки Накладные открывается окно InvoicesWindow (Рисунок 17) со списком всех накладных выбранного склада.



Список накладных

Накладные по складу: Топоры №1

Номер	Дата	Тип	Статус	Позиций	Сумма	
H-0003	30.09.2026 00:00	Приход	Проведена	1	\$30,000.00	
H-0002	29.09.2026 17:32	Приход	Проведена	1	\$15,000.00	
H-0001	29.09.2026 17:32	Приход	Проведена	1	\$10,000.00	

Рисунок-17

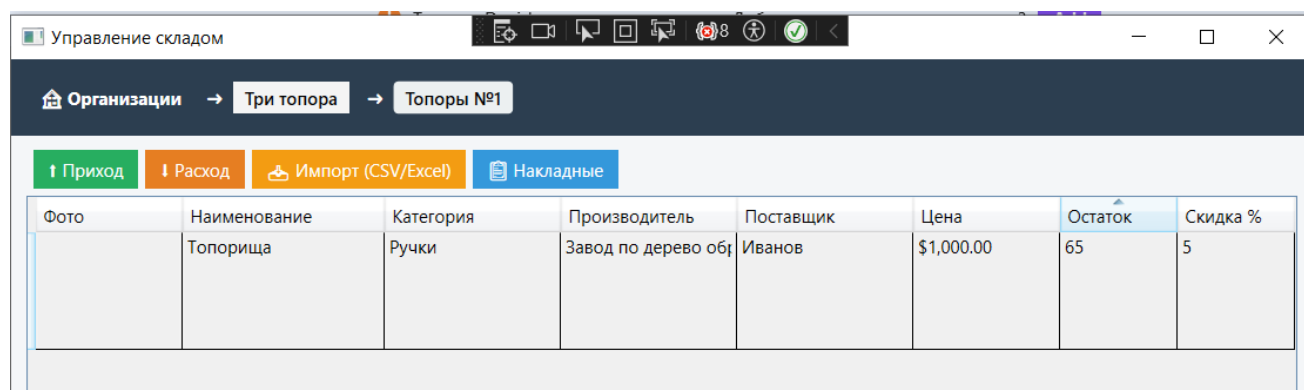
В таблице отображаются: номер, дата, тип, статус, количество позиций, общая сумма.

Просмотр деталей и отмена проведения

Через контекстное меню или кнопки в окне накладных можно:

«Детали» — посмотреть полный состав накладной: наименование товаров, количество, цена, сумма.

«Отменить проведение» — вернуть остатки к исходным значениям. Статус накладной меняется на «Отменена» (Рисунок 18).



Управление складом

Организации → Три топора → Топоры №1

↑ Приход ↓ Расход Импорт (CSV/Excel) Накладные

Фото	Наименование	Категория	Производитель	Поставщик	Цена	Остаток	Скидка %
	Топорища	Ручки	Завод по дерево об	Иванов	\$1,000.00	65	5

Список накладных

Накладные по складу: Топоры №1

Номер	Дата	Тип	Статус	Позиций	Сумма	
H-0003	30.09.2026 00:00	Приход	Проведена	1	\$30,000.00	
H-0002	29.09.2026 17:32	Приход	Проведена	1	\$15,000.00	
H-0001	29.09.2026 17:32	Приход	Готово			

Управление складом

Организации → Три топора → Топоры №1

↑ Приход

↓ Расход

Импорт (CSV/Excel)

Накладные

Фото	Наименование	Категория	Производитель	Поставщик	Цена	Остаток	Скидка %
	Топорища	Ручки	Завод по дерево об	Иванов	\$1,000.00	50	5

Рисунок-18

Написание Unit-тестов

Переходим к проекту Sklad.Tests. В нём находятся шесть файлов:

- **MSTestSettings.cs** — служебный файл, настраивающий фреймворк MSTest. Содержит директиву [assembly: DoNotParallelize] — она отключает параллельное выполнение тестов, чтобы избежать гонок данных в общем синглтоне DataService.
- **MockDialogService.cs** — заглушка интерфейса IDialogService;
- **DataServiceTests.cs** — тесты для хранилища данных;
- **RelayCommandTests.cs** — тесты для класса RelayCommand;
- **ViewModelTests.cs** — тесты для главной ViewModel и ViewModel организаций и товаров;
- **WarehouseViewModelTests.cs** — отдельный файл тестов для ViewModel складов.

Всего написано 31 юнит-тест, покрывающих основные механики приложения.

MockDialogService.cs

Класс MockDialogService реализует интерфейс IDialogService, но вместо открытия диалоговых окон возвращает заранее заданные значения. Например, ShowOrganizationDialog() возвращает true и подставляет в модель имя «Mocked Org», ShowProductDialog() — «Mocked Product», ShowWarehouseDialog() — заполняет поля Name и Address склада. Свойства ConfirmResult, OrganizationDialogResult, ProductDialogResult, WarehouseDialogResult, InvoiceDialogResult позволяют тестам симулировать отказ пользователя.

MSTestSettings.cs

Служебный файл, генерируемый Visual Studio. Содержит одну директиву:

```
csharp
```

```
[assembly: DoNotParallelize]
```

Тесты запускаются последовательно — это гарантирует изоляцию каждого теста в условиях общего синглтона DataService.

DataServiceTests.cs

Содержит три теста для класса DataService:

- **Instance_ShouldBeSingleton** — проверяет, что при двух обращениях к DataService.Instance возвращается один и тот же экземпляр.
- **ResetInstance_ShouldGiveNewInstance** — после ResetInstance() создаётся новый экземпляр.
- **ResetInstance_NewInstanceHasFreshData** — после сброса новый экземпляр получает чистый набор данных из конструктора.

Метод [TestInitialize] Setup() вызывает DataService.ResetInstance() перед каждым тестом.

RelayCommandTests.cs

Содержит пять тестов для класса RelayCommand:

- Execute_ShouldInvokeAction — при вызове Execute() выполняется переданный делегат.
- CanExecute_NullPredicate_ReturnsTrue — без предиката команда всегда доступна.
- CanExecute_UsesPredicate — с предикатом доступность зависит от условия.
- RaiseCanExecuteChanged_ShouldFireEvent — при вызове метода срабатывает событие CanExecuteChanged.
- Constructor_NullAction_Throws — при передаче null в конструктор выбрасывается ArgumentNullException.

ViewModelTests.cs

Содержит 12 тестов для главной ViewModel, ViewModel организаций и товаров. Метод [TestInitialize] Setup() сбрасывает DataService, очищает коллекции и создаёт изолированный набор данных.

Навигация (MainViewModel):

- MainViewModel_InitialState_ShouldBeOrganizationViewModel — при старте открывается модуль организаций.
- NavigateToWarehouses_WithSelectedOrg_ShouldChangeView — при выбранной организации происходит переход к складам.
- NavigateToWarehouses_WithoutSelectedOrg_ShouldNotChangeView — без выбранной организации переход не выполняется.

CRUD организаций:

- OrganizationViewModel_Add_ShouldAddNewOrg — добавление увеличивает количество.
- OrganizationViewModel_Delete_ShouldRemoveOrg — удаление убирает организацию.
- OrganizationViewModel_DeleteCancelled_ShouldKeepOrg — при отмене удаления организация остаётся.
- OrganizationViewModel_Select_ShouldNavigateToWarehouses — при выборе происходит переход к складам.

CRUD товаров:

- ProductViewModel_Add_ShouldAddNewProduct — добавление товара увеличивает количество.
- ProductViewModel_Delete_ShouldRemoveProduct — удаление убирает товар.

Фильтрация:

- ProductViewModel_ShouldFilterByWarehouse — в список попадают только товары выбранного склада.

DataService:

- DataService_ShouldInitializeTestData — после Setup() в сервисе ровно одна организация, один склад, один товар.

WarehouseViewModelTests.cs

Содержит 11 тестов для ViewModel складов:

- Add_WithValidName_ShouldAddWarehouse — добавление склада с валидным именем.
- Add_WithEmptyName_ShouldNotAddWarehouse — пустое имя не добавляется.
- Edit_WithValidName_ShouldChangeName — переименование склада.
- Edit_WithEmptyName_ShouldKeepOldName — пустое значение не перезаписывает старое.
- Delete_WithConfirmation_ShouldRemoveWarehouse — удаление с подтверждением.
- Delete_WithCancellation_ShouldKeepWarehouse — при отмене склад остаётся.
- Select_ShouldNavigateToProducts — при выборе склада происходит переход к товарам.
- Back_ShouldNavigateToOrganizations — возврат к организациям.
- WarehouseViewModel_ShouldFilterWarehousesByOrganization — в список попадают только склады выбранной организации.

Проект Kklad.Tests после создания всех файлов выглядит так (Рисунок 19).

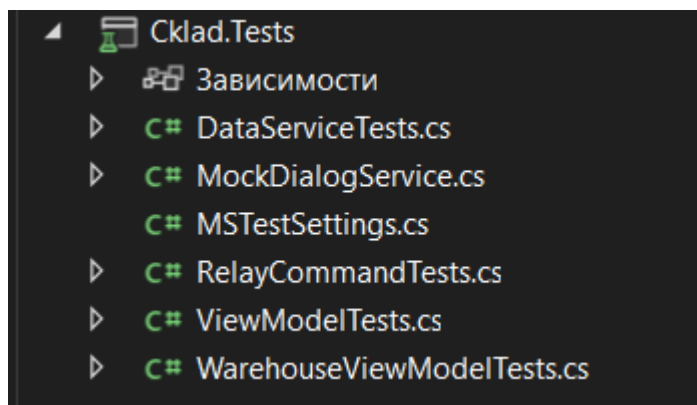


Рисунок-19

Запустим тесты: «Тест» → «Запустить все тесты» (Рисунок 20).

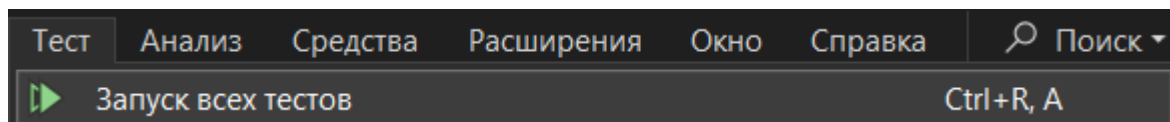


Рисунок-20

Откроется Обзорщик тестов — нажимаем «Выполнить всё». Все 31 тест должны быть с зелёной галочкой (Рисунок 21).

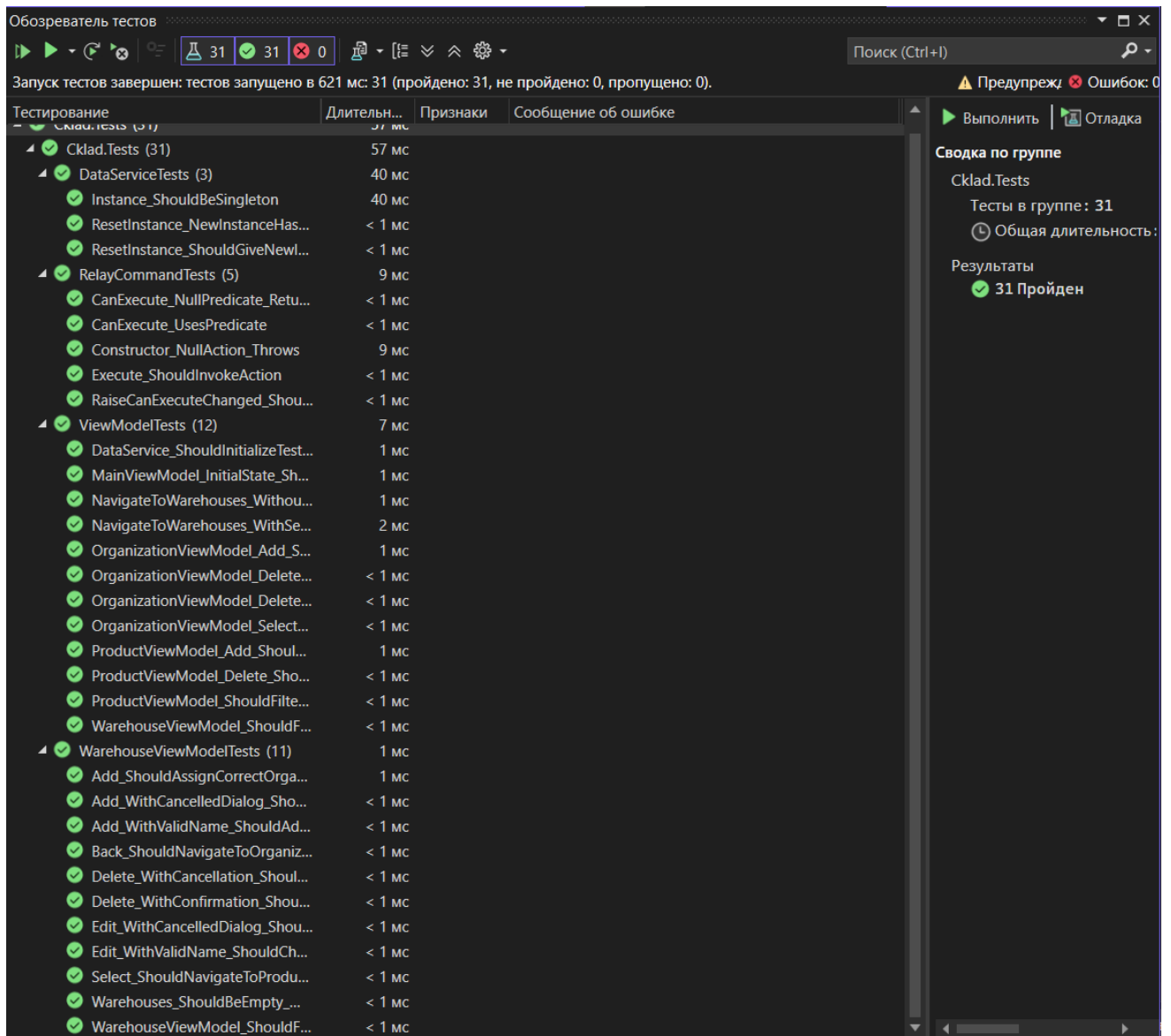


Рисунок-21

Запуск приложения и его тестирование

Нажимаем F5. Приложение запустилось. Открылось главное окно со списком организаций (Рисунок 22).

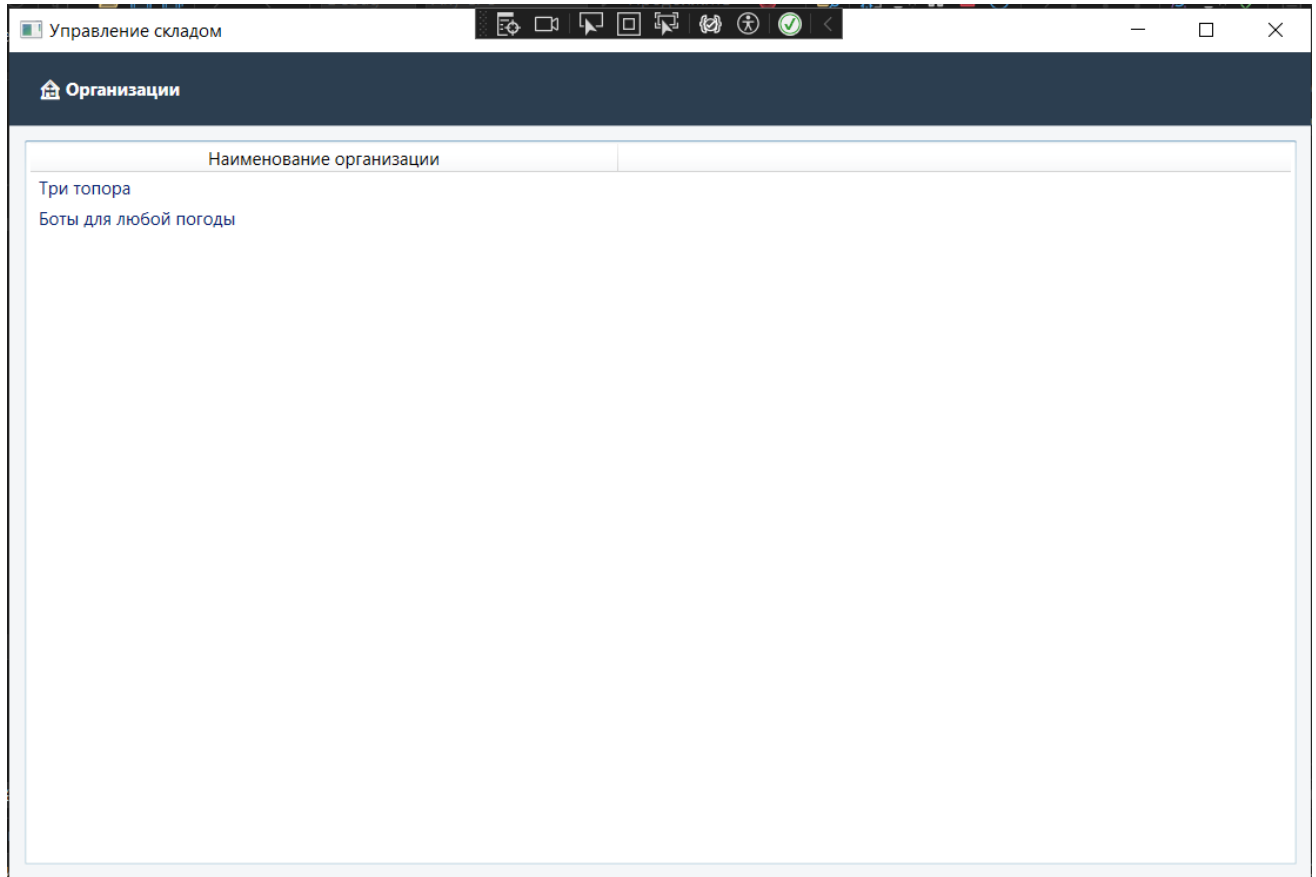


Рисунок-22

Через ПКМ по списку организаций вызываем контекстное меню: «Открыть склады», «Новая организация», «Изменить», «Удалить» (Рисунок 23).

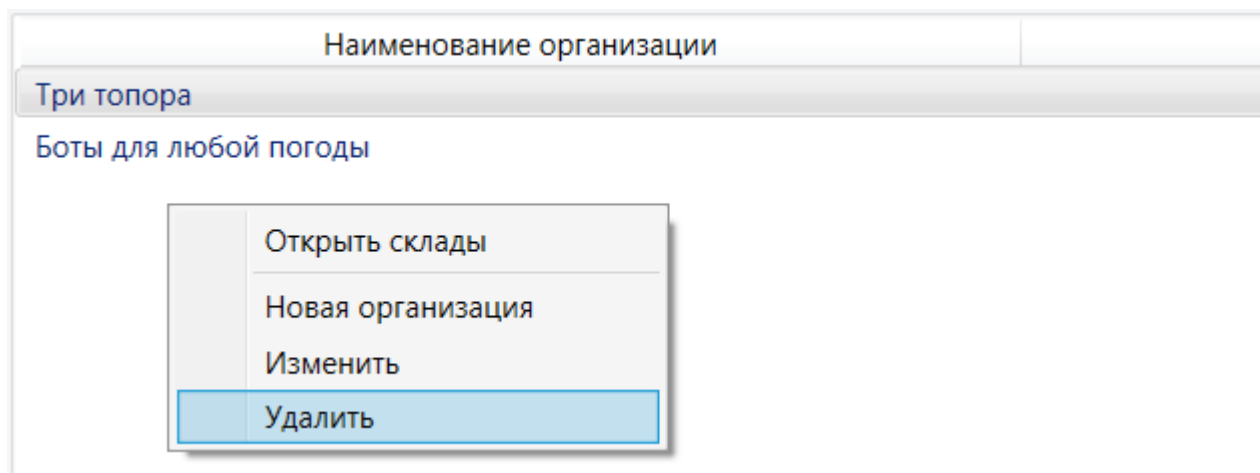


Рисунок-23

Выбираем «Новая организация» — открывается диалог с полем ввода (Рисунок 25).

Организации

Наименование компании:

OK Отмена

Рисунок-24

ПКМ по организации переходим к списку складов (Рисунок 25). Через ПКМ → «Новый склад» открываем диалог склада с двумя полями: «Название склада» и «Адрес» (Рисунок 26).

Организации → Три топора	
Название склада	Адрес
Топоры №1	г. Москва, ул. Ленина, д. 1
Топоры №2	г. Москва, ул. Мира, д. 5

Рисунок-25

Склад

Название склада:

Адрес:

OK Отмена

Рисунок-26

ПКМ по складу открываем таблицу товаров (Рисунок 27). Панель навигации показывает полный путь.

Организации → Три топора → Топоры №1							
↑ Приход ↓ Расход Импорт (CSV/Excel) Накладные							
Фото	Наименование	Категория	Производитель	Поставщик	Цена	Остаток	Скидка %
	Топорища	Ручки	Завод по дерево обра	Иванов	\$1,000.00	50	5

Рисунок-27

В таблице товаров отображаются все колонки, предусмотренные техническим заданием: Фото, Наименование, Категория, Производитель, Поставщик, Цена, Остаток, Скидка %.

ПКМ по товару открываем диалог редактирования (Рисунок 28).

Товар

2

Наименование*:

Топорища

Фото...

Категория:

Ручки

Производитель:

Завод по дерево обработке

Поставщик:

Иванов

Цена*:

1000

Остаток:

50

OK

Отмена

Рисунок-28

При попытке сохранить товар с пустым наименованием появляется сообщение об ошибке валидации (Рисунок 29).

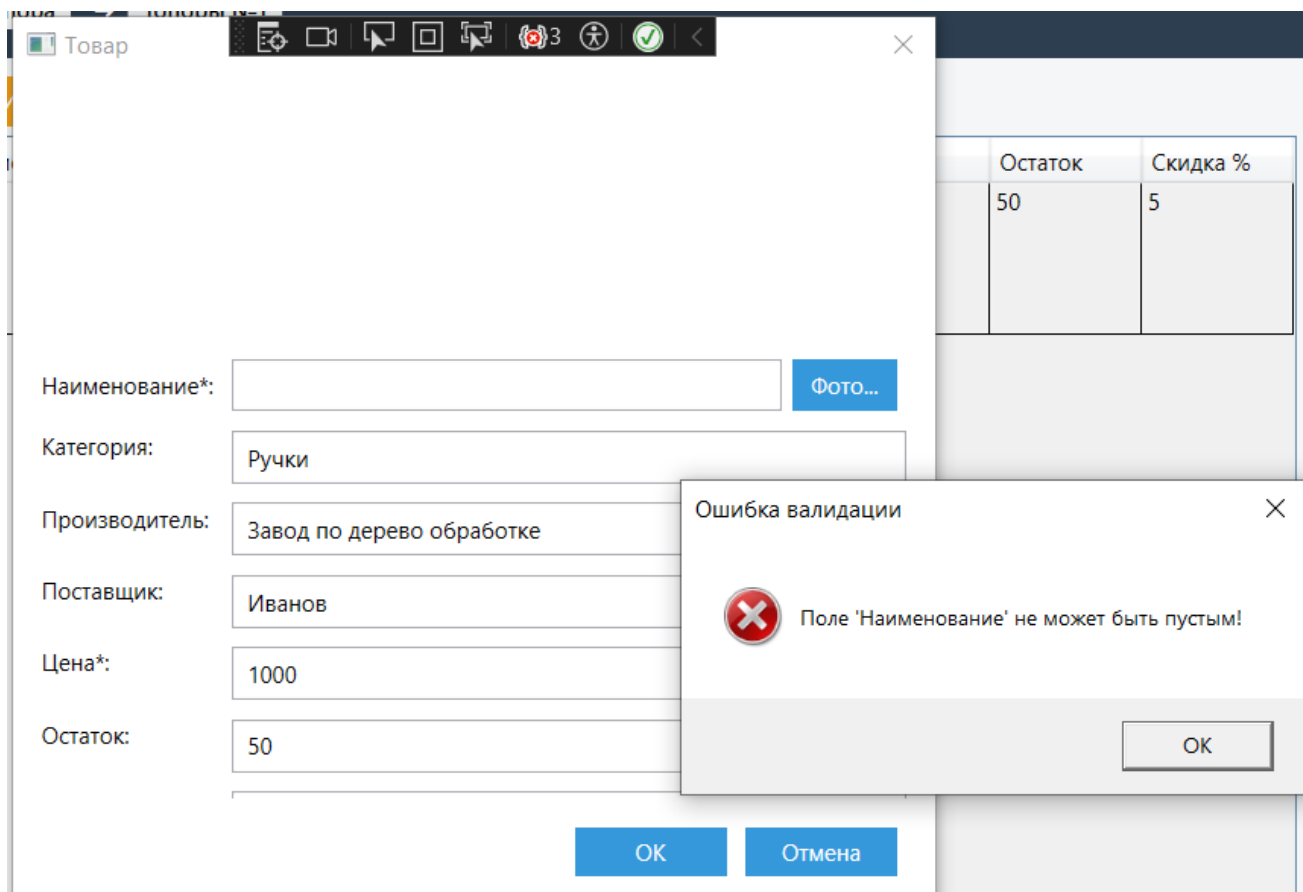


Рисунок-29

На панели товаров находятся четыре кнопки (Рисунок 30).

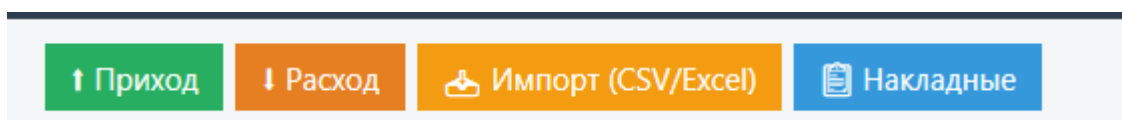


Рисунок-30

Кнопка « Приход»

Нажимаем « Приход» — открывается диалог накладной прихода (Рисунок 31). Добавляем позицию, указываем товар и количество. После ОК остатки увеличиваются, накладная сохраняется.

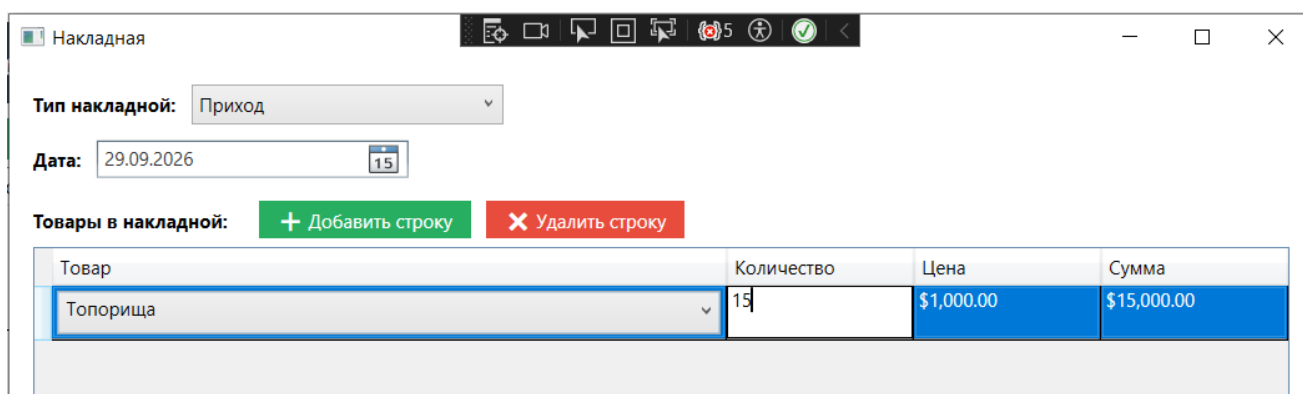


Рисунок-31

Результат виден в таблице — остаток увеличился, накладная появилась в списке (Рисунок 32).

↑ Приход ↓ Расход Импорт (CSV/Excel) Накладные							
Фото	Наименование	Категория	Производитель	Поставщик	Цена	Остаток	Скидка %
	Топорища	Ручки	Завод по дерево об	Иванов	\$1,000.00	65	5

Рисунок-32

Кнопка «Расход»

Нажимаем «Расход» — открывается окно списания товаров. Если количество превышает остаток, появится предупреждение «Недостаточно товара» (Рисунок 33).

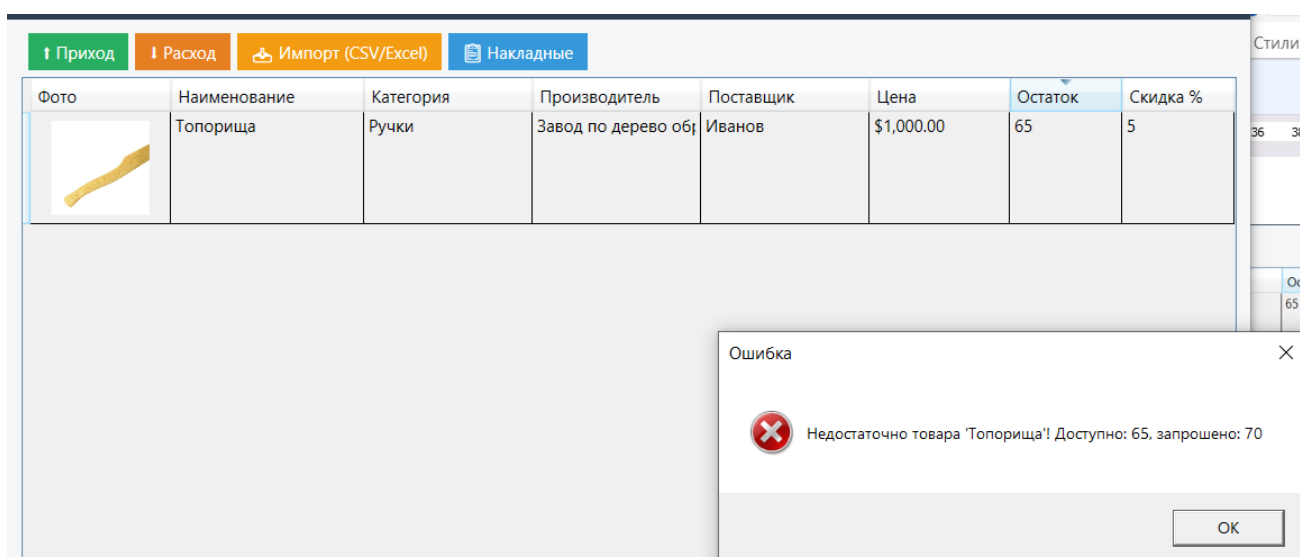


Рисунок-33

Кнопка «Импорт (CSV/Excel)»

Реализован импорт товаров из CSV и Excel. Готовим CSV-файл с семью колонками (Рисунок 34).



Рисунок-34

Нажимаем кнопку «Импорт (CSV/Excel)» — выбираем файл. Товары появляются в таблице, а на экране отображается сообщение «Успешно импортировано товаров: 5» (Рисунок 35).

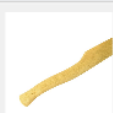
Фото	Наименование	Категория	Производитель	Поставщик	Цена	Остаток	Скидка %
	Топорища	Ручки	Завод по дерево об	Иванов	\$1,000.00	65	5
	Отвёртка крестовая	Инструмент	Wera	ПрофИнструмент	\$320.00	40	0
	Молоток слесарный	Инструмент	Зубр	СтройМаркет	Импорт завершён  Успешно импортировано: 5 ОК		
	Болгарка 125мм 900	Электроинструмент	DeWalt	ТехноТрейд			
	Дрель-шуруповёрт а	Электроинструмент	Makita	ТехноТрейд	\$8,990.00	12	10
	Перфоратор SDS-Plu	Электроинструмент	Bosch	ТехноТрейд	\$12,400.00	8	0

Рисунок-35

Кнопка «Накладные»

Нажимаем кнопку «Накладные» — открывается окно InvoicesWindow со списком всех накладных склада (Рисунок 36).

Накладные по складу: Топоры №1

Номер	Дата	Тип	Статус	Позиций	Сумма	
H-0001	29.09.2026 00:00	Приход	Проведена	1	\$15,000.00	

Рисунок-36

Двойной клик по накладной (или ПКМ → «Детали») показывает полный состав документа (Рисунок 37).

Накладные по складу: Топоры №1

Номер	Дата	Тип	Статус	Позиций	Сумма	
H-0001	29.09.2026 00:00	Приход	Проведена	1	\$15,000.00	

Детали накладной



Накладная: H-0001

Дата: 29.09.2026 00:00

Тип: Приход

Статус: Проведена

Товар	Кол-во	Цена	Сумма
Топорища	15	1 000,00 Р	15 000,00 Р

Итого: 15 000,00 Р

ОК

Рисунок-37

Через ПКМ → «Отменить проведение» можно отменить накладную — остатки вернутся к исходным значениям, а статус изменится на «Отменена». (Рисунок 38-39)

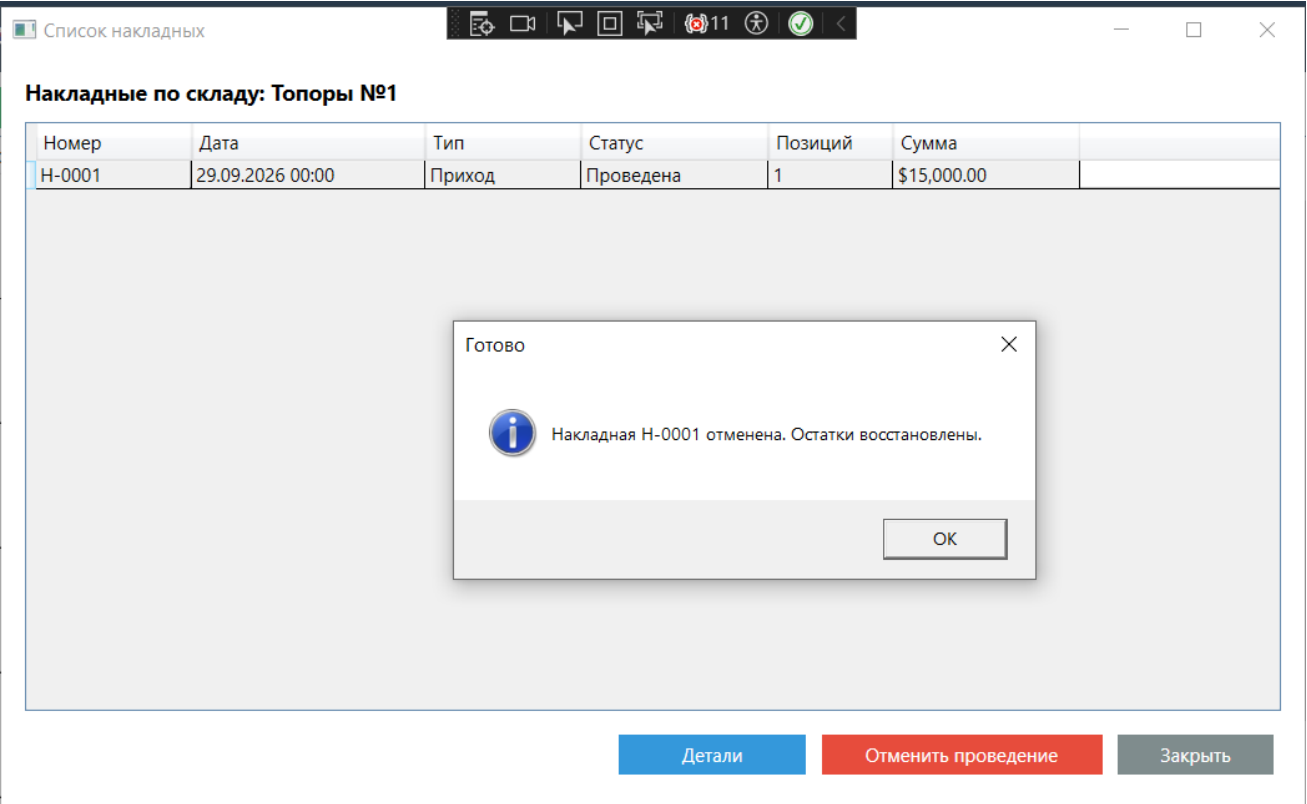


Рисунок-38

Фото	Наименование	Категория	Производитель	Поставщик	Цена	Остаток	Скидка %
	Топорища	Ручки	Завод по дерево об	Иванов	\$1,000.00	50	5

Рисунок-39

Заключение

В ходе выполнения практической работы было разработано WPF-приложение «Управление складом». Приложение реализовано на платформе .NET Framework 4.8 с использованием языка C#, графического фреймворка WPF и архитектурного паттерна MVVM.

В результате работы были решены следующие задачи:

- Спроектирована структура данных — модели Organization, Warehouse, Product, Invoice и InvoiceItem собраны в единый файл Entities.cs.
- Создана библиотека классов Sклад.Core, содержащая всю бизнес-логику приложения. Это обеспечивает чистоту архитектуры и возможность тестирования логики независимо от UI.

Реализованы сервисы для работы с данными:

- DataService — синглтон-хранилище коллекций организаций, складов, товаров и накладных;
- IDialogService — интерфейс абстракции диалоговых окон.

Реализованы ViewModels для всех модулей: MainViewModel, OrganizationViewModel, WarehouseViewModel, ProductViewModel, а также вспомогательные классы ViewModelBase и RelayCommand.

Созданы пять диалоговых окон: OrganizationDialog, WarehouseDialog, ProductDialog, InvoiceDialog, InvoicesWindow. Каждое окно содержит форму с валидацией и кнопки действий.

Реализовано управление приложением преимущественно мышью:

- ПКМ — контекстное меню действий;
- панель навигации в верхней части окна — переход между уровнями иерархии;
- четыре функциональные кнопки — Приход, Расход, Импорт (CSV/Excel), Накладные.

Реализован полный документооборот накладных:

- создание с автоматической генерацией номера (Н-0001, Н-0002);
- проверка достаточности товара при расходе;
- сохранение накладных в истории;
- просмотр списка и деталей;
- отмена проведения с восстановлением остатков.

Реализована валидация вводимых данных — наименование не может быть пустым, цена должна быть положительным числом, скидка — в диапазоне от 0 до 100%.

Написаны Unit-тесты (31 тест) на базе фреймворка MSTest, распределённые по шести файлам. Тесты покрывают навигацию, фильтрацию по контексту, CRUD-операции, работу команд и работу синглтона. Все тесты успешно проходят.

- Реализован импорт товаров из файлов CSV и Excel.
- Реализован выбор фотографии товара через стандартный диалог OpenFileDialog.

Архитектурные преимущества:

- Логика приложения полностью отделена от UI и находится в библиотеке Sklad.Core.
- Тесты зависят только от библиотеки и не запускают WPF.
- Класс DialogService изолирует работу с окнами в отдельном сервисе.
- Соблюдается принцип единственной ответственности — каждый класс отвечает за свою задачу.
- Управление мышью и минимальное количество кнопок делают интерфейс интуитивно понятным.

Таким образом, все поставленные задачи выполнены в полном объёме, приложение работает и соответствует требованиям технического задания.

Список использованных источников

Троелсен Э. Язык программирования C# 7 и платформы .NET и .NET Core. — М.: Диалектика, 2018.

Нейгел К., Ивсен Б. C# 5.0 и платформа .NET 4.5 для профессионалов. — М.: Диалектика, 2013.

Официальная документация Microsoft: <https://learn.microsoft.com/ru-ru/dotnet/>

Официальная документация по WPF: <https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/>

Официальный сайт ClosedXML: <https://github.com/ClosedXML/ClosedXML>